

Sorting

March 11, 2024

Selection Sort

- ▶ Say, the elements to sort occupy positions 0 to $n - 1$ of an array.
- ▶ for $i = 0$ to $n - 2$
 - ▶ /* The elements in positions upto $i - 1$ are sorted. */
 - ▶ Find the smallest among the elements in positions i to $n - 1$.
 - ▶ Swap it with the i -th element.

Time taken = $n + (n - 1) + \dots + 2 = O(n^2)$.

Selection Sort - Example Walkthrough

Initial Array: [64, 25, 12, 22, 11]

Iteration 1: Select 11 (min) and swap with the 1st element.

Array: [11, 25, 12, 22, 64]

Iteration 2: Select 12 (min) and swap with the 2nd element.

Array: [11, 12, 25, 22, 64]

Iteration 3: Select 22 (min) and swap with the 3rd element.

Array: [11, 12, 22, 25, 64]

Iteration 4: Select 25 (min) and swap with the 4th element.

Sorted Array: [11, 12, 22, 25, 64]

Selection Sort: C code

```
#include <stdio.h>

void swap(int v[], int i, int j) {
    int temp;
    temp = v[i]; v[i] = v[j]; v[j] = temp;
}

void selectionSort(int arr[], int n) {
    int i, j, minIndex;

    for (i = 0; i < n-1; i++) {
        minIndex = i;
        for (j = i+1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swap(arr, i, minIndex);
    }
}
```

Selection Sort: C code

```
#include <stdio.h>

void printArray(int arr[], int size) {
    int i;
    for (i=0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Original array: \n");
    printArray(arr, n);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);

    return 0;
}
```

Insertion Sort

- ▶ Say, the elements to sort occupy positions 0 to $n - 1$ of an array.
- ▶ for $i = 1$ to $n - 1$
 - ▶ /* The elements in positions 0 to $i - 1$ are sorted. */
 - ▶ Pick the i -th element k out of the array.
 - ▶ Scanning right to left, move the previous elements to the right by one position, until an element less than k is reached.
 - ▶ Insert k to the right of it.

Time taken = $1 + 2 + \dots + (n - 1) = O(n^2)$.

Insertion Sort: Example

- ▶ Original array : 12, 11, 13, 5, 6
- ▶ After the first pass : 11, 12, 13, 5, 6
- ▶ After the second pass : 11, 12, 13, 5, 6
- ▶ After the third pass : 5, 11, 12, 13, 6
- ▶ After the fourth pass : 5, 6, 11, 12, 13

Insertion Sort: C code

```
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Insertion Sort: Example

```
int main() {  
    int arr[] = {12, 11, 13, 5, 6};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("Original array: \n");  
    printArray(arr, n);  
  
    insertionSort(arr, n);  
  
    printf("Sorted array: \n");  
    printArray(arr, n);  
  
    return 0;  
}
```

Bubble Sort

- ▶ say, the elements to sort occupy positions 0 to $n - 1$ in an array.
- ▶ imagine the array vertically with position 0 at the bottom.
- ▶ for $i = 0$ to $n - 2$
 - ▶ /* move the $(n-i)$ -th smallest to the $(n-i-1)$ -st place */
 - ▶ for $j = 0$ to $n - i - 2$
 - ▶ if the j -th element and the next are out of order, swap them.

Time taken = $(n - 1) + (n - 2) + \dots + 1 = O(n^2)$.

Bubble Sort-Example: **Original Array:** 64, 25, 12, 22, 11

		1	2	3	4	5	6	7	8	9	10
4	11	11	11	11	64	64	64	64	64	64	64
3	22	22	22	64	11	11	11	25	25	25	25
2	12	12	64	22	22	22	25	11	11	22	22
1	25	64	12	12	12	25	22	22	22	11	12
0	64	25	25	25	25	12	12	12	12	12	11

Pass 1: Columns 1, 2, 3, 4

Pass 2: Columns 5, 6, 7

Pass 3: Columns 8, 9

Pass 4: Column 10

Bubble Sort - C Code

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr, j, j + 1);
            }
        }
    }
}
```

Bubble Sort - C Code

```
int main() {  
    int arr[] = {64, 25, 12, 22, 11};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("Original array: ");  
    printArray(arr, n);  
  
    bubbleSort(arr, n);  
  
    printf("Sorted array: ");  
    printArray(arr, n);  
  
    return 0;  
}
```

Counting Sort

- ▶ Sorting algorithm introduced by Harold H. Seward in 1954.
- ▶ Operates in linear time.
- ▶ Efficient for sorting integers with a known range.

Counting Sort

1. Find the range of input elements.
2. Create an auxiliary array (count array) to store element counts.
3. Traverse the input array and update the count array.
4. Compute the prefix sums over the count array.
5. for $i = n - 1$ downto 0
 - ▶ Let j be the i -th input element
 - ▶ There are $\text{count}[j]$ input elements that are $\leq$ this j ; so, this j goes to output location $\text{count}[j] - 1$.
 - ▶ decrement $\text{count}[j]$.

Counting Sort

- ▶ Time complexity: $O(n + k)$
- ▶ n - number of elements in the array
- ▶ k - range of input values
- ▶ Stable sort - maintains relative order of equal elements between input and output

Counting Sort

- ▶ Advantages:
 - ▶ Linear time complexity for suitable cases.
 - ▶ Stable sorting.
- ▶ Limitations:
 - ▶ Practical for small ranges, less practical for large ranges.
 - ▶ Limited to integer data.

Example

- ▶ Consider the input array: $[4_a, 2_a, 3_a, 1_a, 4_b, 2_b, 3_b, 4_c, 1_b, 2_c]$.
- ▶ The subscripts are to distinguish elements with the same key values
- ▶ Range: 1 to 4. (subscripts are not part of keys)
- ▶ Count array initialization: $[0, 0, 0, 0]$.

Step 1: Count Occurrences

- ▶ Traverse the input and update the count array.
- ▶ Input array: $[4_a, 2_a, 3_a, 1_a, 4_b, 2_b, 3_b, 4_c, 1_b, 2_c]$.
- ▶ Count array: $[2, 3, 2, 3]$.

Step 2: Cumulative Counts

- ▶ Compute prefix sums over the count array.
- ▶ Prefix sums of the count array: $[2, 5, 7, 10]$.

Step 3: Build Sorted Output

- ▶ Use the prefix sums array to build the sorted output array.
- ▶ 2_c is the last element; there are 5 that are $\leq 2_c$; so, 2_c goes to output location 4.
- ▶ Prefix sums array is $[2, 4, 7, 10]$ now.
- ▶ 1_b is the 2nd last element; there are 2 that are $\leq 1_b$; so, 1_b goes to output location 1.
- ▶ Prefix sums array is $[1, 4, 7, 10]$ now.
- ▶ 4_c is the 3rd last element; there are 10 that are $\leq 4_c$; so, 4_c goes to output location 9.
- ▶ Prefix sums array is $[1, 4, 7, 9]$ now.
- ▶ ...
- ▶ Sorted output array: $[1_a, 1_b, 2_a, 2_b, 2_c, 3_a, 3_b, 4_a, 4_b, 4_c]$.

Counting Sort - C Code

```
#include <stdio.h>

void countingSort(int arr[], int n) {
    // Find the maximum and minimum values in the array
    int max = arr[0], min = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
        if (arr[i] < min) {
            min = arr[i];
        }
    }

    // Create the count array and initialize to 0
    int range = max - min + 1;
    int count[range];
    for (int i = 0; i < range; i++) {
        count[i] = 0;
    }
}
```

Counting Sort - C Code

```
// Count the occurrences of each element
for (int i = 0; i < n; i++) {
    count[arr[i] - min]++;
}
// Update the count array with prefix sums
for (int i = 1; i < range; i++) {
    count[i] += count[i - 1];
}
// Create the sorted output array
int output[n];
for (int i = n - 1; i >= 0; i--) {
    output[count[arr[i] - min] - 1] = arr[i];
    count[arr[i] - min]--;
}
// Copy the sorted output back to the original array
for (int i = 0; i < n; i++) {
    arr[i] = output[i];
}
}
```

Counting Sort - C Code

```
int main() {  
    int arr[] = {4, 2, 1, 7, 5, 3, 8, 6};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("Original array: ");  
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
  
    countingSort(arr, n);  
  
    printf("\nSorted array: ");  
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
  
    return 0;  
}
```

Radix Sort

- ▶ Radix sort is a non-comparative sorting algorithm.
- ▶ It operates by distributing elements into buckets based on individual digits or radix.

Algorithm Overview

1. **Initialization:** Determine the maximum number of digits in the given sequence of numbers.
2. **Passes:** Perform passes through the digits from LSD to MSD or vice versa.
3. **Counting Sort:** Apply Counting Sort to the current sequence based on the current digit being processed.
4. **Repeat:** Output of one pass is input to the next.

Variants of Radix Sort

- ▶ **LSD Radix Sort:** Starts sorting from the rightmost digit.
(Preferred.)
- ▶ **MSD Radix Sort:** Starts sorting from the leftmost digit.
- ▶ MSD Radix Sort gives 10 sequences corresponding to MSD 0, 1, ..., 9.
- ▶ Each has to be sorted recursively and combined.
- ▶ LSD Radix sort has only one sequence to handle all the time.

LSD Radix Sort

- ▶ E.g., Stack of 1000 answer papers to sort with rollnumbers from 000 to 999
- ▶ Form 10 buckets corresponding to digits 0 to 9
- ▶ Place papers into buckets face down based on LSD; 247 goes to bucket-7
- ▶ Flip the buckets face up; stack bucket-0 on bucket-1 on bucket-2 etc.
- ▶ Use 2nd LSD for the next pass, which also end with one stack.
- ▶ Repeat.

Performance

- ▶ Radix sort is effective for sorting integers or fixed-length strings.
- ▶ Linear time complexity makes it efficient for large datasets.
- ▶ Performance can degrade with numbers having varying numbers of digits.

Example

- ▶ **Initial array:**
- ▶ 170, 048, 075, 090, 802, 024, 002, 066, 273, 300, 497, 189, 921
- ▶ **Pass 1:** Sort by the units place.
- ▶ 170, 090, 300, 921, 802, 002, 273, 024, 075, 066, 497, 048, 189
- ▶ **Pass 2:** Sort by the tens place.
- ▶ 300, 802, 002, 921, 024, 048, 066, 170, 273, 075, 189, 090, 497
- ▶ **Pass 3:** Sort by the hundreds place.
- ▶ 002, 024, 048, 066, 075, 090, 170, 189, 273, 300, 497, 802, 921

Performance and Complexity

- ▶ Linear time complexity for integers with fixed-length.
- ▶ Degrades with varying numbers of digits.

Radix Sort - C Code

```
#include <stdio.h>

// Function to find the maximum number
// to determine the number of digits
int getMax(int array[], int n) {
    int max = array[0];
    for (int i = 1; i < n; i++)
        if (array[i] > max)
            max = array[i];
    return max;
}
```

Radix Sort - C Code

```
// Using counting sort to sort the elements
// based on significant places
void countingSort(int array[], int n, int exp) {
    const int RANGE = 10; // Assuming base 10 (decimal system)
    int output[n];
    int count[RANGE] = {0};
        // Count occurrences in count[]
    for (int i = 0; i < n; i++)
        count[(array[i] / exp) % RANGE]++;
        // Cumulative count to get actual position
    for (int i = 1; i < RANGE; i++)
        count[i] += count[i - 1];
        // Build the output array
    for (int i = n - 1; i >= 0; i--) {
        output[count[(array[i] / exp) % RANGE] - 1] = array[i];
        count[(array[i] / exp) % RANGE]--;
    }
        // Copy the output array to the original array
    for (int i = 0; i < n; i++)
        array[i] = output[i];
}
```

Radix Sort - C Code

```
// Radix Sort function
void radixSort(int array[], int n) {
    // Find the maximum number to determine the number of digits
    int max = getMax(array, n);

    // Perform counting sort for every digit place
    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSort(array, n, exp);
}

// Print an array
void printArray(int array[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", array[i]);
    printf("\n");
}
```

Radix Sort - C Code

```
int main() {  
    int array[] = {170, 45, 75, 90, 802, 24, 2, 66};  
    int n = sizeof(array) / sizeof(array[0]);  
  
    printf("Original array: ");  
    printArray(array, n);  
  
    // Perform radix sort  
    radixSort(array, n);  
  
    printf("Sorted array: ");  
    printArray(array, n);  
  
    return 0;  
}
```

Recursion

- ▶ C supports recursive functions.
- ▶ Example 1: printing a number as a character string.
- ▶ Example 2: quicksort algorithm.
- ▶ Recursion is especially convenient for recursively defined data structures.

Recursion Example: Printing a Number

```
#include <stdio.h>

void printd(int n) {
    if (n < 0) {
        putchar('-');
        n = -n;
    }
    if (n / 10)
        printd(n / 10);
    putchar(n % 10 + '0');
}
```

Recursive Quicksort Example

```
void qsort(int v[], int left, int right) {  
    int i, last;  
    void swap(int v[], int i, int j);  
  
    if (left >= right)  
        return;  
  
    swap(v, left, (left + right) / 2);  
    last = left;  
  
    for (i = left + 1; i <= right; i++)  
        if (v[i] < v[left])  
            swap(v, ++last, i);  
  
    swap(v, left, last);  
  
    qsort(v, left, last - 1);  
    qsort(v, last + 1, right);  
}
```

Recursive Quicksort Example

```
/* swap: interchange v[i] and v[j] */  
void swap(int v[], int i, int j)  
{  
    int temp;  
  
    temp = v[i];  
    v[i] = v[j];  
    v[j] = temp;  
}
```

Merge Sort Overview

- ▶ Divide and conquer algorithm
- ▶ Splits the array into two halves
- ▶ Recursively sorts each half
- ▶ Merges the sorted halves

Merge

```
void merge(int arr[], int l, int m, int r) {  
    int n1 = m - l + 1;  
    int n2 = r - m;  
    int L[n1], R[n2];  
  
    for (int i = 0; i < n1; ++i) L[i] = arr[l + i];  
    for (int j = 0; j < n2; ++j) R[j] = arr[m + 1 + j];  
  
    int i = 0, j = 0, k = l;  
  
    while (i < n1 && j < n2)  
        if (L[i] <= R[j]) arr[k++] = L[i++];  
        else arr[k++] = R[j++];  
  
    while (i < n1) arr[k++] = L[i++];  
    while (j < n2) arr[k++] = R[j++];  
}
```

Merge Sort

```
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = l + (r - l) / 2;  
  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
  
        merge(arr, l, m, r);  
    }  
}
```

Merge Sort

```
int main() {  
    int arr[] =  
        {38, 27, 43, 3, 9, 82, 10, 18,  
         75, 32, 6, 89, 47, 17, 57, 69};  
    int arr_size = sizeof(arr) / sizeof(arr[0]);  
  
    mergeSort(arr, 0, arr_size - 1);  
  
    // Display sorted array  
    for (int i = 0; i < arr_size; ++i)  
        printf("%d ", arr[i]);  
  
    return 0;  
}
```

Merge Sort

- ▶ Merge Sort is an efficient and stable sorting algorithm.
- ▶ Time complexity: $O(n \log n)$
- ▶ Space complexity: $O(n)$

Static Variables

- ▶ Variables like `sp` and `val` in `stack.c`, and `buf` and `bufp` in `getch.c`, are private to their respective source files.
- ▶ `static` declaration for external variables or functions limits their scope to the source file being compiled.
- ▶ External `static` hides names, providing a way to keep them private.
- ▶ Example: `static char buf[BUFSIZE]; static int bufp = 0;`
- ▶ External `static` can be applied to functions, making their names invisible outside the file.

Static Variables Example

```
static char buf[BUFSIZE];  
static int bufp = 0;  
  
int getch(void) { ... }  
  
void ungetch(int c) { ... }
```

Internal Static Variables

- ▶ Internal `static` variables are local to a function but persist across function calls.
- ▶ They provide private, permanent storage within a single function.
- ▶ Example: `static int count = 0;`

Internal Static Variables Example

```
void exampleFunction() {  
    static int count = 0;  
    // count retains its value across calls  
    // other local variables are recreated each time  
}
```

Register Variables

- ▶ Register declaration advises the compiler that the variable will be heavily used.
- ▶ Intended for variables to be placed in machine registers for potential speedup.
- ▶ Limited number of variables in each function may be kept in registers.
- ▶ Specific restrictions on types and numbers of register variables vary across machines.
- ▶ Compiler may ignore the register advice.

Register Variables Example

```
register int x;  
register char c;  
  
f(register unsigned m, register long n) {  
    register int i;  
    // ...  
}
```

Block Structure

- ▶ C is not block-structured like Pascal.
- ▶ Variables can be defined within blocks.
- ▶ Block-scoped variables hide outer block variables.
- ▶ Automatic variables in a block are re-initialized upon block entry.
- ▶ Automatic variables hide external variables and functions of the same name.

Block Structure Example

```
if (n > 0) {  
    int i; /* declare a new i */  
    for (i = 0; i < n; i++)  
        ...  
}
```

The scope of `i` is the “true” branch of the `if` ; this `i` is unrelated to any `i` outside the block.

Block Structure Example

```
int x;  
int y;  
  
f(double x)  
{  
    double y;  
}
```

Within `f`, occurrences of `x` refer to the parameter, which is a double; outside, they refer to the external `int`.

Initialization

- ▶ External and static variables are guaranteed to be initialized to zero if not explicitly initialized.
- ▶ Automatic and register variables have undefined initial values (garbage).
- ▶ Scalar variables may be initialized with definition.
- ▶ For external and static variables, the initializer must be a constant expression. Initialization is done once
- ▶ For automatic and register variables, any expression using previously defined values, even function calls can be the initializer.

Initialization Examples

```
int x = 1;
char squota = '\';
long day = 1000L * 60L * 60L * 24L; /* milliseconds/day */

int days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

char pattern[] = "ould";
           /* equivalent to {'o', 'u', 'l', 'd', '\0'} */
```

The C Preprocessor

File Inclusion

- ▶ File inclusion simplifies the handling of collections of `#defines` and declarations.
- ▶ Any source line of the form:
 - ▶ `#include "filename"`
 - ▶ `#include <filename>`is replaced by the contents of the file `filename`.
- ▶ If the filename is quoted, searching typically begins where the source program was found.
- ▶ If not found there, or if the name is enclosed in `<` and `>`, an implementation-defined rule is followed to find the file.
- ▶ An included file may itself contain `#include` lines.

The C Preprocessor

File Inclusion

- ▶ Commonly, several `#include` lines are used at the beginning of a source file.
- ▶ They include common `#define` statements and extern declarations.
- ▶ They also access function prototype declarations for library functions from headers like `<stdio.h>`.
- ▶ This is the preferred way to tie declarations together for a large program.
- ▶ It guarantees that all source files will be supplied with the same definitions and variable declarations, avoiding bugs.

The C Preprocessor

File Inclusion

- ▶ When an included file is changed, all files that depend on it must be recompiled.
- ▶ Header files need not be actual files; details are implementation-dependent.

The C Preprocessor

Macro Substitution

- ▶ A definition has the form
`#define name replacement-text.`
- ▶ Macro substitution replaces subsequent occurrences of the token `name` with the `replacement-text`.
- ▶ The scope of a name defined with `#define` is from its point of definition to the end of the source file.
- ▶ Definitions may use previous definitions, and substitutions are made only for tokens.
- ▶ Substitutions do not take place within quoted strings.

The C Preprocessor

Macro Substitution

```
#define forever for (;;)
/* infinite loop */
```

```
#define max(A, B) ((A) > (B) ? (A) : (B))
x = max(p+q, r+s);
/* Expands: x = ((p+q)>(r+s) ? (p+q) : (r+s)); */
/* Expressions evaluated twice; side effects! */
```

```
#define square(x) x * x      /* square(z+1) */
/* parentheses? order of evaluation issues */
```

The C Preprocessor

Macro Substitution

- ▶ Despite pitfalls, macros are valuable for performance and flexibility.
- ▶ Eg. from `<stdio.h>`: `getchar` and `putchar` defined as macros to avoid run-time overhead of function call per character processed.
- ▶ Names can be undefined with `#undef` to ensure consistency.

The C Preprocessor

Macro Substitution, # operator

```
#define dprint(expr) printf(#expr " = %g\n", expr)
```

```
dprint(x/y)  
/* Expands into: printf("x/y" " = %g\n", x/y); */  
/* Equivalent to: printf("x/y = %g\n", x/y); */
```

The C Preprocessor

Macro Substitution, ## operator

```
#define paste(front, back) front ## back
```

```
paste(name, 1)  
/* Expands into: name1 */
```

The C Preprocessor

Conditional Inclusion

- ▶ Conditional inclusion allows selective inclusion of code during compilation.
- ▶ `#if` evaluates a constant integer expression.
- ▶ If the expression is non-zero, subsequent lines until `#endif`, `#elif`, or `#else` are included.
- ▶ The expression `defined(name)` in a `#if` is 1 if the name has been defined, and 0 otherwise.

The C Preprocessor

Conditional Inclusion

```
#if !defined(HDR)
#define HDR
/* contents of hdr.h go here */
#endif
/* included only once */
```

The C Preprocessor

Conditional Inclusion

```
#if SYSTEM == SYSV
#define HDR "sysv.h"
#elif SYSTEM == BSD
#define HDR "bsd.h"
#elif SYSTEM == MSDOS
#define HDR "msdos.h"
#else
#define HDR "default.h"
#endif
#include HDR
```

The C Preprocessor

Conditional Inclusion

```
/* another way */  
#ifndef HDR  
#define HDR  
/* contents of hdr.h go here */  
#endif
```